



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

◆ Analisi del codice **LogReader**

1. Scopo generale

Il file `LogReader.vb` è una classe **utility** (non ereditabile) che fornisce metodi statici per gestire un file di log di nome `ItalPascal_Log.txt`.

Questa classe permette di:

- Leggere l'intero contenuto del log.
- Aprirlo con Notepad.
- Cancellarlo (eliminare il file).

Tutto il lavoro è svolto in modo **thread-safe** e con gestione degli errori tramite `Try/Catch`.

2. Componenti chiave

Sezione	Descrizione	Funzione principale
<code>Imports</code>	Importa i namespace necessari (<code>System.IO</code> , <code>System.Diagnostics</code> , <code>System.Windows.Forms</code>).	Fornisce accesso a file, processi e UI.
<code>Public NotInheritable Class LogReader</code>	Classe finale, non estendibile.	Contiene tutti i metodi di utilità.
<code>Private Sub New()</code>	Costruttore privato.	Impedisce l'istanza della classe (tutti i metodi sono statici).
<code>Private Shared ReadOnly LogFile As String</code>	Costante che costruisce il percorso completo del file di log.	Centralizza la posizione del file.

Sezione	Descrizione	Funzione principale
Public Shared Function ReadLog() As String	Legge l'intero file di log.	Restituisce il contenuto o un messaggio di errore.
Public Shared Sub OpenLog()	Avvia Notepad con il file di log.	Permette all'utente di visualizzare il log in un editor di testo.
Public Shared Sub ClearLog()	Elimina il file di log se esiste.	Pulisce lo spazio di archiviazione.
Vedi LogLeggiScrivi	Suggerimenti su come usare la classe.	Fornisce esempi di utilizzo.

Elemento	Funzione
Path.Combine(...)	Unisce in modo sicuro più segmenti di percorso, aggiungendo i separatori di directory corretti per il sistema operativo
Application.StartupPath	Restituisce la cartella in cui è stato avviato l'eseguibile (es. C:\Programmi\ItalPascal\)
"ItalPascal_Log.txt"	Specifica il nome del file che verrà creato o scritto

3. Flusso di esecuzione

3.1 Costruzione del percorso del file

```

1 Private Shared ReadOnly LogFile As String =
2     Path.Combine(
3         Application.StartupPath,
4         "ItalPascal_Log.txt")

```

- `Application.StartupPath` restituisce la cartella in cui è stato avviato l'applicazione.
- `Path.Combine` crea un percorso completo e sicuro, indipendentemente dal sistema operativo.

3.2 Lettura del log (`ReadLog`)

1. **Verifica esistenza:** `If File.Exists(LogFile)` .
2. **Lettura:** `File.ReadAllText(LogFile)` restituisce il contenuto come stringa.
3. **Fallback:** Se il file non esiste, ritorna `"Nessun log disponibile."` .
4. **Gestione eccezioni:** In caso di errore, restituisce `ex.Message` .

3.3 Apertura del log (`OpenLog`)

1. **Verifica esistenza:** `If File.Exists(LogFile) .`
2. **Avvio Notepad:** `Process.Start("notepad.exe", LogFile) .`
3. **Messaggio di errore:** Se il file non è presente, mostra un `MessageBox .`
4. **Gestione eccezioni:** Mostra eventuali errori con `MessageBox.Show(ex.Message) .`

3.4 Cancellazione del log (`ClearLog`)

1. **Verifica esistenza:** `If File.Exists(LogFile) .`
 2. **Eliminazione:** `File.Delete(LogFile) .`
 3. **Gestione eccezioni:** Mostra eventuali errori con `MessageBox.Show(ex.Message) .`
-

4. Come usarla

```
1  ' Leggi e mostra il log
2  MessageBox.Show(LogReader.ReadLog(), "FILE LOG")
3
4  ' Apri il log con Notepad
5  LogReader.OpenLog()
6
7  ' Cancella il log
8  LogReader.ClearLog()
```

◆ Conclusioni

Questa classe è ideale per piccoli progetti in cui si desidera un semplice sistema di logging senza dipendenze esterne.

Per applicazioni più complesse, si consiglia di utilizzare librerie di logging (es. NLog, log4net) che offrono funzionalità avanzate.